MIT 8103 Advanced Database Systems

IsaacMart database portfolio summary

Idowu Isaac | Student ID 30168852 | September Cohort 2026

Preparation status

AI-assisted practical portfolio with student-run verification in GitHub Codespaces. Student evidence was committed and pushed as bf95174. Assessor access and final submission checks remain pending.

Case study and scope

IsaacMart is a simulated Nigerian e-commerce retailer. Customers order products, warehouse staff manage stock, finance staff reconcile payments and managers analyse sales. PostgreSQL stores authoritative transactions in NGN; MongoDB holds flexible catalogue attributes. All records are synthetic. Delivery, taxes and refunds are outside this exercise.

Portfolio 1 Database design and modelling

The relational model has Customers, Categories, Products, Orders, OrderItems and Payments. Customer-to-order, category-to-product and order-to-payment relationships are one-to-many; OrderItems resolves the order/product many-to-many relationship. Primary keys, foreign keys, unique email/SKU/payment references and CHECK constraints protect row-level integrity. The model is in 3NF under the stated dependencies. Historical unit_price belongs to the sale line; current price belongs to Products. Totals are derived. Cross-row payment reconciliation still requires application checks.

Evidence and data

The design folder contains executable schema/seed SQL, the ER diagram and setup output. The generated dataset contains 10,000 customers, 1,000 products, 200,000 orders, 200,000 line items and 180,000 successful payments. Detailed rules and exclusions appear in the activity README.

Portfolio 2 Query processing and optimisation

The customer-history query filters customer_id and returns the latest ten orders. A B-tree on (customer_id, ordered_at DESC) supports filtering and ordering. Median server execution time across five measured runs after one warm-up was 12.802 ms before and 0.093 ms after indexing (137.66x observed ratio). Raw JSON plans and timings are retained. Cache state, sequential conditions and one local container limit generalisation; index storage and write costs remain. Other queries cover category sales, monthly sales, customers without orders and payment reconciliation. In my Codespace verification, a single comparison measured 13.326 ms before and 0.131 ms after indexing, with the plan changing from a parallel sequential scan and sort to an index scan. These student timings are distinct from the assistant-run five-measurement medians.

Portfolio 3 Transactions and concurrency

A committed stock decrement changes 100 to 98; a later five-unit decrement is rolled back, retaining 98. A separate checkout demonstration groups stock reservation, order, line and pending payment, then rolls back. External payment settlement requires idempotency and an outbox/saga; the database cannot atomically commit a remote gateway call.

Two actual PostgreSQL sessions demonstrate row locking: session A holds a product update while B reaches a 500ms lock timeout. A rolls back. A conditional UPDATE with stock >= requested quantity prevents stale read-then-write overselling when callers check the returned row count. REPEATABLE READ evidence shows session A reading 100 twice while B commits 101; after A commits, its new query reads 101. READ COMMITTED instead obtains a fresh statement snapshot. Deadlock/serialization failures require retries.

Portfolio 4 NoSQL and advanced models

MongoDB embeds category-specific attributes in product documents: headphones have battery hours, shirts have size/material, and lamps have watts. The script inserts three documents, queries battery hours, updates an attribute and verifies unique-SKU and schema-validation rejection. PostgreSQL owns price and stock; MongoDB owns descriptive attributes. An outbox-based synchronisation design is discussed but not implemented. No cross-store foreign-key or transaction guarantee is claimed.

Portfolio 5 Distributed and cloud databases

The discussion proposes a managed PostgreSQL primary with a cross-zone standby and asynchronous reporting replicas, alongside a MongoDB replica set. Synchronous acknowledgements trade latency/write availability for reduced loss exposure; asynchronous replicas may lag. Fenced failover must prevent split-brain writers. Browse descriptions may tolerate staleness, while checkout reads authoritative stock. Replication does not replace encrypted backups and restoration drills.

Customer-based sharding could co-locate orders but complicates shared product inventory and cross-shard checkout. Date partitioning supports retention and does not by itself provide multi-server sharding. Optimisation and scale-up should precede sharding. This component follows the brief's discussion option: no cloud deployment, replication, failover or recovery measurements are claimed.

Verification evidence

The repository contains initial assistant-run outputs and separate Student-prefixed evidence from my Codespace. Student checks cover setup counts, query results, index plans, commit/rollback, locking, repeatable-read observations, MongoDB validation and SQL constraints. The guided isolation sequence was also reviewed through terminal output shared in chat; its full first-session transcript is not in the saved final-state file.

Repository and submission review

Repository: https://github.com/LaProfessor/Idowu_Isaac_30168852_MIT8103 . Student-provided terminal output confirms evidence commit bf95174 was pushed to master. The assessor must receive and accept access to the private repository before submission. Commit dates reflect actual work; no semester-long development history is claimed.

Individual technical reflection

I learned that an index can substantially reduce the work needed for a selective query. In my customer-history test, execution time fell from 13.326 ms with a parallel sequential scan and sort to 0.131 ms with an index scan, approximately a 102-fold improvement in this single comparison. The composite B-tree helped PostgreSQL locate one customer’s orders in date order and stop after ten rows. These timings are specific to my test; indexes also require storage and write maintenance.

The two-session exercise showed that an uncommitted update can block another update to the same row. The second session reached its two-second lock timeout while the first held the lock. Rolling back the first transaction restored product 1 stock to 98. In the repeatable-read test, one transaction continued to see stock 100 after another session committed 101; a new query after commit saw 101.

Saving evidence from the container was the most tedious part. A relative output path failed inside psql because the repository folders were outside the database container. With guidance, I exited psql and saved command output from the Codespace shell using tee. I also had to keep the first transaction open while using a second terminal. These steps helped me distinguish database transactions from the surrounding container and terminal workflow. I would improve the exercise by repeating the performance measurements and comparing additional query filters.

AI use declaration

I used OpenAI ChatGPT/Codex to help select the case study, generate and explain the initial schema, scripts, synthetic data, diagram, analysis and documentation, and guide my verification. The assistant also ran initial tests in its own environment. I subsequently executed the supplied setup, queries, index comparison, commit/rollback, two-session lock timeout, repeatable-read isolation, MongoDB catalogue and integrity checks in my GitHub Codespace, and shared the outputs for review. My query execution times were 13.326 ms before indexing and 0.131 ms afterward in a single comparison. I saved student-run evidence and pushed it in commit bf95174. The assistant helped edit my reflection and correct technical wording. The distributed/cloud component is a design discussion; I did not deploy or test a distributed cluster. I remain responsible for reviewing, understanding and explaining the submitted work.

References

Miva University, MIT 8103 Advanced Database Systems CA September Cohort 2026, supplied brief. PostgreSQL 17 documentation: constraints, EXPLAIN, multicolumn indexes, transaction isolation, explicit locking and warm standby (postgresql.org/docs/17/). MongoDB documentation: data modelling, schema validation and replication (mongodb.com/docs/manual/). Full section URLs appear in References.txt. Synthetic data was generated for this exercise.

Before the single submission

Upload this revised summary, reflection and AI declaration to the private repository. Confirm assessor access, accepted submission formats and the deadline. Folder: Idowu_Isaac_30168852. Filenames retain the student prefix. The similarity index has not been measured; the LMS maximum is 20%, excluding bibliography.